

- JavaScript is a lightweight, interpreted, multi-paradigm high-level programming language.
- Modern JavaScript engines use Just-In-Time (JIT) compilation for performance.
- It runs inside web browsers and also on servers through runtimes like Node.js, making it suitable for both frontend and backend development.

## Why is JavaScript So Popular?

- Runs in every modern browser.
- Can be used for both frontend and backend.
- Huge ecosystem (npm).
- Large community support.
- Easy to learn.
- Supports asynchronous programming.
- Cross-platform.

## ECMAScript (ES)

- ECMAScript (ES) is the specification that defines how JavaScript should work.
- It specifies the language syntax, features, behavior, and rules that JavaScript engines must implement.

## Why was ECMAScript created?

- In the early days, different browsers implemented JavaScript differently this caused compatibility issues because the same JavaScript code could behave differently across browsers.
- To solve this, **ECMA International** standardized the language to ECMAScript

| ECMAScript                       | JavaScript                           |
|----------------------------------|--------------------------------------|
| Standard / Specification         | Programming Language                 |
| Defines language rules           | Implements those rules               |
| Maintained by ECMA International | Implemented by browsers and runtimes |
| Cannot execute code              | Executes code                        |

## ES6 (ECMAScript 2015)

- ES6, also known as ECMAScript 2015, is a major version of the ECMAScript specification that introduced features like `let` , `const` , arrow functions, classes, modules, template literals, and promises.
- 

## JavaScript Engine

- A JavaScript engine is a software program that compiles and executes JavaScript code by converting it into machine code.

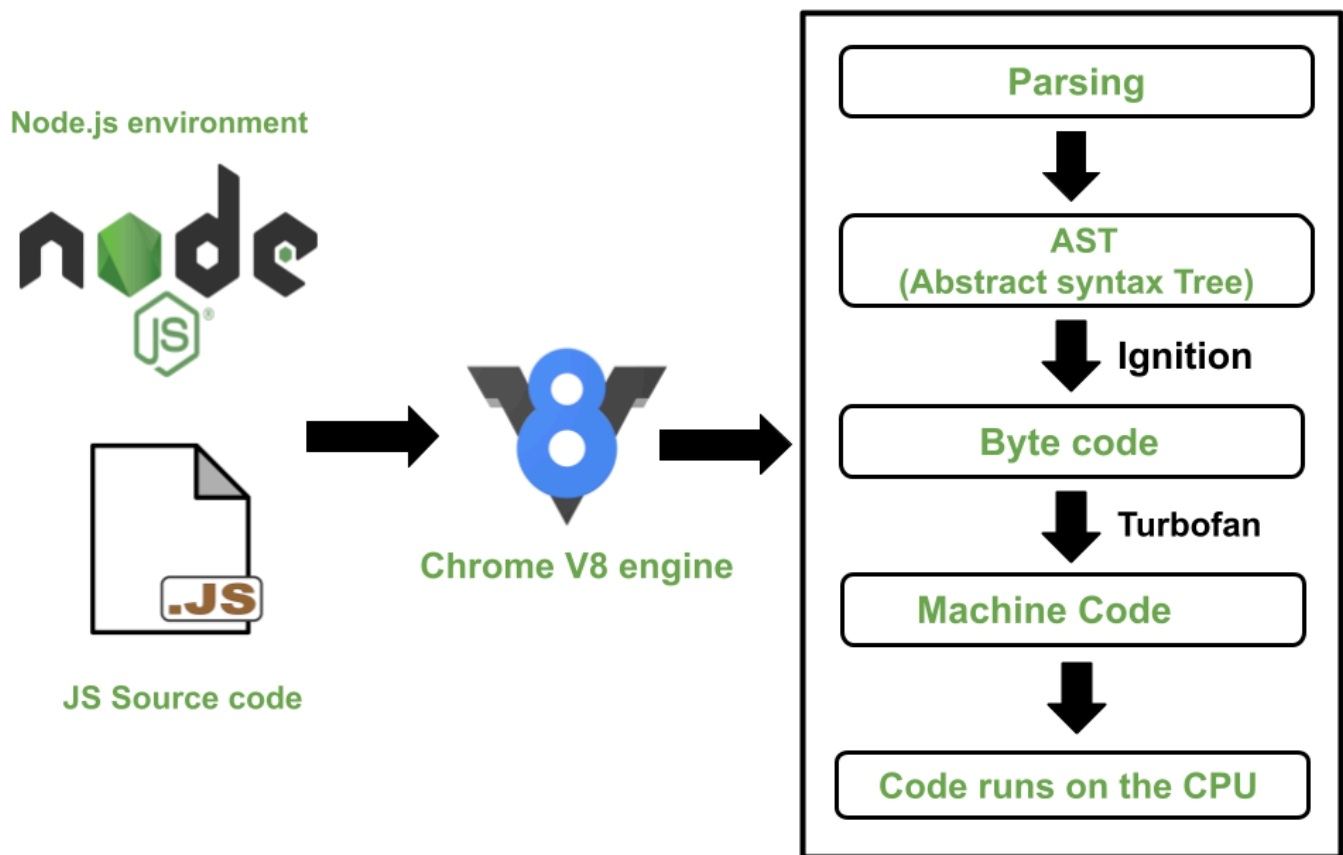
### Popular JavaScript Engines

| Engine         | Developed By | Used In                         |
|----------------|--------------|---------------------------------|
| V8             | Google       | Chrome, Node.js                 |
| JavaScriptCore | Apple        | Safari                          |
| Chakra         | Microsoft    | Legacy Internet Explorer / Edge |

## V8

- V8 is Google's open-source JavaScript engine that powers Google Chrome, Node.js and Electron (VS Code, Discord, Slack).
- It executes JavaScript by using Just-In-Time (JIT) compilation to convert JavaScript into optimized machine code at runtime.
- **Node.js** provides the runtime environment and additional APIs for server-side development.

### Working of V8 Engine



- The JavaScript source code is parsed, where it is tokenized, checked for syntax errors, and converted into an **Abstract Syntax Tree (AST)**.
- The **AST** is passed to **Ignition** (V8's interpreter), which converts it into **Bytecode**. Ignition then starts executing the bytecode.
- While the program is running, **TurboFan** identifies **frequently executed (hot)** bytecode and compiles it into **optimized machine code**.
- The **CPU** executes the optimized machine code instead of interpreting the bytecode, resulting in faster execution.

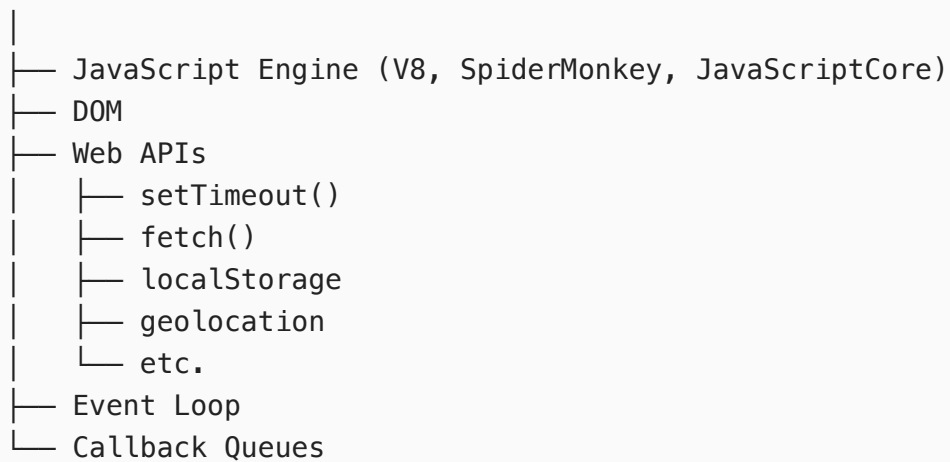
**Bytecode** is an intermediate representation between JavaScript source code and machine code that enables faster startup and efficient execution.

## Browser Runtime

- A browser runtime is the environment provided by a web browser that allows JavaScript to execute.

- It consists of the JavaScript engine along with browser-provided APIs such as the DOM, Web APIs, and the Event Loop.

#### Browser Runtime

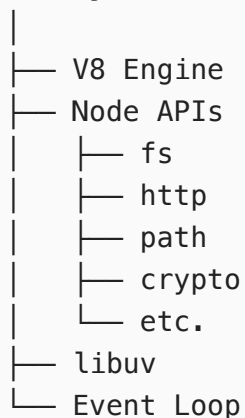


---

## Node.js Runtime

- Node.js runtime is an environment that allows JavaScript to run outside the browser.
- It embeds the V8 JavaScript engine and provides server-side APIs such as the File System, HTTP, and networking modules.

#### Node.js Runtime



---

## Compilation & Interpretation

### Interpretation

Interpretation is the process of executing code line by line without producing a standalone executable beforehand.

---

## Compilation

Compilation is the process of translating source code into machine code before execution.

## Is JavaScript interpreted or compiled?

Modern JavaScript is **neither purely interpreted nor purely compiled**. Engines like V8 use **Just-In-Time (JIT) compilation**, where code is first interpreted as bytecode and frequently executed code is compiled into optimized machine code.

---

## Strict Mode ( "use strict" )

- Strict Mode is a JavaScript feature that enables a stricter set of rules, helping catch common coding mistakes and preventing certain unsafe behaviours.

```
"use strict";
```

```
"use strict"
```

```
x=10;
```

```
console.log(x); // throws ReferenceError, Preventing accidental global  
variable declaration
```

---

## Variables

- A variable is a named memory location used to store data that can be accessed and manipulated during program execution.
- JavaScript provides three ways to declare variables:
  - `var`
  - `let`
  - `const`

## var

- `var` is the original way of declaring variables in JavaScript. It is function-scoped, can be redeclared, reassigned, and is hoisted with an initial value of `undefined`.

```
var name = "Pavan";

name = "John";    // Reassignment

var name = "Bob"; // Redeclaration
```

## let

- `let` declares a block-scoped variable that can be reassigned but cannot be redeclared in the same scope.

```
let age = 22;

age = 23;    // Reassignment

let age = 30; // Redeclaration = SyntaxError
```

## const

- `const` declares a block-scoped variable whose binding cannot be reassigned after initialization.

```
const PI = 3.14;

PI = 10;    // Error
```

---

# Variable Scope

- Scope defines the region of a program where a variable is accessible.

## Global Scope

```
let name = "Pavan";

function greet() {
```

```
    console.log(name);  
}
```

## Function Scope

```
function test() {  
  
    var x = 10;  
  
}  
  
console.log(x); // ❌
```

## Block Scope

```
if (true) {  
  
    let a = 10;  
  
    const b = 20;  
  
}  
  
console.log(a); // ❌
```

---

## Hoisting

- Hoisting is JavaScript's behavior of moving declarations to memory before code execution begins.

### Function Declarations

- Traditional function declarations are fully hoisted, meaning you can call them safely before they are written in the file.

```
greet(); // Outputs: "Hello!"  
  
function greet() {  
    console.log("Hello!");  
}
```

```
}
```

## Variable Hoisting with `var`

- When you use `var`, JavaScript reserves memory and gives it a placeholder value of `undefined`

```
console.log(user); // Outputs: undefined

var user = "Alice";
console.log(user); // Outputs: "Alice"
```

## Lexical Hoisting with `let` and `const`

- Variables declared with `let` and `const` are technically hoisted, but the engine does not initialize them with a default value.
- They enter a **Temporal Dead Zone (TDZ)** from the moment the scope opens until the line where they are declared is executed.

```
console.log(age); // Throws ReferenceError: Cannot access 'age' before
initialization

let age = 25;
```

## Function Expressions and Arrow Functions

- Functions stored inside variables are treated exactly like regular variable assignments.

```
sayHi(); // Throws TypeError: sayHi is not a function (because sayHi is
currently 'undefined')

var sayHi = function() {
  console.log("Hi!");
};
```

---

## Temporal Dead Zone (TDZ)



- The Temporal Dead Zone (TDZ) is the period between entering a scope and the point where a `let` or `const` variable is declared, during which accessing the variable throws a `ReferenceError`.

```
console.log(age); // Throws ReferenceError: Cannot access 'age' before
initialization
```

```
let age = 25;
```

```
console.log(age); // ReferenceError: Cannot access 'age' before initialization
// ^—— TDZ starts here (beginning of the block/scope)
// 'age' exists in memory but is uninitialized and cannot be accessed TDZ
//
```

```
let age = 25; // Declaration + Initialization happens here
              // TDZ ends here
console.log(age); // 25 Valid Zone
// From this point until the end of the block/scope,
// 'age' is initialized and can be safely accessed
```

---

## Global Object

- The global object is the top-level object provided by the JavaScript runtime that contains globally available properties and functions.
- Browser → `window`
- Node.js → `global`

```
var x = 10;
```

```
console.log(window.x);
```

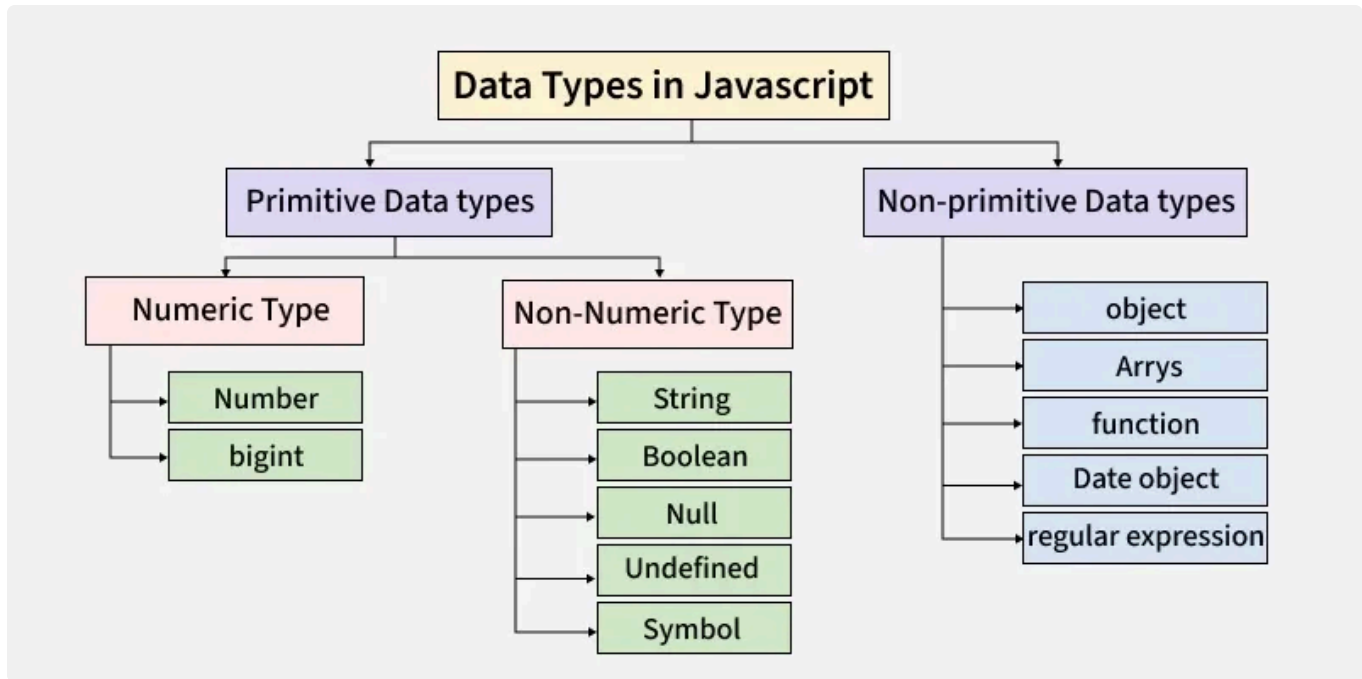
- Because `var` becomes a property of the global object (in browsers).

---

## Data Types

- A data type defines the kind of value a variable can store and determines the operations that can be performed on that value

```
let age = 25;           // Number
let name = "Pavan";    // String
let isActive = true;   // Boolean
```



## Primitive Types

- Primitive data types store a single immutable value.
- Variables hold the actual value, not a reference to another object.

### 1. Number

- The `Number` type represents both integer and floating-point numeric values.

```
let age = 25;
let price = 99.99;
let temperature = -5;
```

### 2. String

- A `String` represents a sequence of characters enclosed in single quotes, double quotes, or backticks.

```
let name = "Pavan";  
let city = 'Bangalore';  
let message = `Hello`;
```

### 3. Boolean

- A `Boolean` represents one of two logical values: `true` or `false`.

```
let isLoggedIn = true;  
let isAdmin = false;
```

### 4. Undefined

- `undefined` is the default value assigned to a declared variable that has not been initialized.

```
let age;  
  
console.log(age);
```

### 5. Null

- `null` represents the absence of any object value.

```
let user = null;
```

### 6. Symbol

- A `Symbol` is a unique and immutable primitive value primarily used as an object property key to avoid naming conflicts.

```
const id1 = Symbol("id");  
  
const id2 = Symbol("id");  
  
console.log(id1 === id2); //false
```

### 7. BigInt

- `BigInt` is used to represent integers larger than the maximum safe integer supported by the `Number` type.

```
const value = 123456789012345678901234567890n;
```

## Reference Types

- Reference types store complex data. Variables hold a reference to the object rather than the object itself.

### 1. Object

- An Object is a collection of key-value pairs used to represent complex entities.

```
const user = {  
  name: "Pavan",  
  age: 22  
};
```

### 2. Array

- An Array is an ordered collection of multiple values stored under a single variable.

```
const nums = [10, 20, 30];
```

### 3. Function

- A Function is a reusable block of code that performs a specific task.
- In JavaScript, functions are first-class objects.

```
function greet() {  
  console.log("Hello");  
}
```

- You can attach properties to a function.

```
function greet() {}  
  
greet.version = "1.0";  
  
console.log(greet.version);
```

### 4. Date

- The `Date` object represents dates and times.

```
const today = new Date();
```

## 5. Map

- Map is a collection of key-value pairs where keys can be of any data type.

```
const map = new Map();  
  
map.set("name", "Pavan");  
map.set(1, "One");
```

## 6. Set

- Set is a collection of unique values. Duplicate values are automatically ignored.

```
const nums = new Set([1, 2, 2, 3]);  
  
console.log(nums);
```

## typeof

- `typeof` is a unary operator that returns a string representing the type of a value or variable.

## Why does `typeof null` return `"object"` ?

- It is a historical bug in JavaScript that has been retained for backward compatibility. Although `typeof null` returns `"object"`, `null` is actually a primitive data type.

```
// Primitive Types  
console.log(typeof 10);           // "number"  
console.log(typeof "Hello");      // "string"  
console.log(typeof true);         // "boolean"  
console.log(typeof undefined);    // "undefined"  
console.log(typeof null);         // "object", Historical bug  
console.log(typeof Symbol());     // "symbol"  
console.log(typeof 123n);         // "bigint"  
  
// Reference Types  
console.log(typeof {});           // "object"  
console.log(typeof []);          // "object"  
console.log(typeof new Date());   // "object"
```

```
console.log(typeof new Map());    // "object"
console.log(typeof new Set());    // "object"
console.log(typeof function(){}); // "function"
```

---

# Operators

## Arithmetic Operators

| Operator | Description         | Example |
|----------|---------------------|---------|
| +        | Addition            | 5 + 2   |
| -        | Subtraction         | 5 - 2   |
| *        | Multiplication      | 5 * 2   |
| /        | Division            | 5 / 2   |
| %        | Modulus (Remainder) | 5 % 2   |
| **       | Exponentiation      | 5 ** 2  |
| ++       | Increment           | x++     |
| --       | Decrement           | x--     |

**Prefix(++x):** Value is updated **before** it is used.

**Postfix(x++):** Value is used first, then updated.

## Assignment Operators

| Operator | Meaning                   |
|----------|---------------------------|
| =        | Assign                    |
| +=       | Add and Assign            |
| -=       | Subtract and Assign       |
| *=       | Multiply and Assign       |
| /=       | Divide and Assign         |
| %=       | Modulus and Assign        |
| **=      | Exponentiation and Assign |

## Comparison Operators

| Operator | Meaning               |
|----------|-----------------------|
| ==       | Loose Equal           |
| ===      | Strict Equal          |
| !=       | Loose Not Equal       |
| !==      | Strict Not Equal      |
| >        | Greater Than          |
| <        | Less Than             |
| >=       | Greater Than or Equal |
| <=       | Less Than or Equal    |

## Logical Operators

| Operator | Meaning     |
|----------|-------------|
| &&       | Logical AND |
| \        | Logical OR  |
| !        | Logical NOT |

## Bitwise Operators

| Operator | Meaning                    |
|----------|----------------------------|
| &        | Bitwise AND                |
| \        | Bitwise OR                 |
| ^        | Bitwise XOR (Exclusive OR) |
| ~        | Bitwise NOT                |
| <<       | Left Shift                 |
| >>       | Right Shift                |

## Ternary Operator

- The ternary operator is a shorthand for an `if...else` statement that evaluates a condition and returns one of two values

```
condition ? expression1 : expression2;
```

```
let age = 20;

let result = age >= 18 ? "Adult" : "Minor";

console.log(result);
```

## Optional Chaining ( ?. )

- Optional chaining safely accesses nested object properties without throwing an error if an intermediate property is `null` or `undefined`.

```
const user = null;

console.log(user?.address?.city); // undefined
```

## Nullish Coalescing ( ?? )

- The nullish coalescing operator returns the right-hand operand only when the left-hand operand is `null` or `undefined`.

```
let username = null;

console.log(username ?? "Guest");
```

## NaN

- NaN (Not-a-Number) is a special numeric value representing an invalid or undefined mathematical operation.

```
console.log(0 / 0); // NaN
console.log(Number("abc")); // NaN

console.log(Number.isNaN(NaN)); // true
```

## Infinity

- Infinity is a special numeric value representing a value greater than any finite number.

```
console.log(10 / 0); // Infinity
console.log(-10 / 0); // -Infinity
```



---

# Function

- A function is a reusable block of code designed to perform a specific task. It executes only when it is invoked (called).

```
function greet() {  
    console.log("Hello");  
}  
  
greet();
```

## Function Declaration

- A Function Declaration defines a named function using the `function` keyword.
- It is hoisted completely, allowing it to be called before its declaration.

```
function functionName(parameters) {  
  
}
```

## Function Expression

- A Function Expression creates a function and assigns it to a variable. Unlike function declarations, function expressions are not fully hoisted.

```
const greet = function () {  
    console.log("Hello");  
};  
  
greet();
```

## Arrow Functions

- An Arrow Function is a shorter syntax for writing functions introduced in ES6.

```
const add = (a, b) => a + b;  
  
// Equivalent to  
function add(a, b) {
```

```
    return a + b;
}
```

## Anonymous Functions

- An anonymous function is a function without a name

```
const greet = function () {
    console.log("Hello");
};
```

## Parameters

- Variables defined in the function.

```
function add(a, b) {}
```

## Arguments

- Actual values passed.

```
add(10, 20);
```

## Default Parameters

- Default parameters provide a default value when an argument is not supplied or is undefined.

```
function greet(name = "Guest") {
    console.log(name);
}

greet();
```

## Rest Parameters

- Rest parameters collect multiple arguments into a single array.

```
function sum(...nums) {

}
```

## arguments Object

- The `arguments` object is an array-like object containing all arguments passed to a regular function
- Not available in Arrow Functions
- `arguments` object is an old feature where as rest parameter is introduced in ES6 to achieve this in simpler way

```
function show() {  
    console.log(arguments);  
}  
  
show(1, 2, 3);
```

## Immediately Invoked Function Expression (IIFE)

- An IIFE is a function that executes immediately after it is defined.

```
(function () {  
    console.log("Executed");  
})();
```

## Callback Functions

- A callback function is a function passed as an argument to another function to be executed later

```
function greet(name) {  
    console.log(name);  
}  
  
function process(callback) {  
    callback("Pavan");  
}  
  
process(greet);
```

## Higher-Order Functions

- A higher-order function is a function that takes another function as an argument,

```
function greet(callback) {
    callback();
}

greet(function () {
    console.log("Hello");
});
```

## Closures

- A closure is the combination of a function and its lexical environment, allowing the function to access variables from its outer scope even after the outer function has finished executing.

```
function outer() {
    let outerVar = "I'm in the outer scope!";

    function inner() {
        console.log(outerVar);
    }
    return inner;
}

const closure = outer();

closure();
closure();
```

## Currying

- Currying is a technique of transforming a function that takes multiple arguments into a sequence of functions, each taking one argument.

```
function multiply(a) {
    return function (b) {
        return a * b;
    };
}

console.log(multiply(2)(5));
```

## Memoization

- Memoization is an optimization technique that stores the results of expensive function calls and returns the cached result for repeated inputs.

```
const cache = {};  
  
function square(n) {  
  if (cache[n]) return cache[n];  
  
  cache[n] = n * n;  
  
  return cache[n];  
}
```

---

## Objects

- An object is a reference data type that stores data as a collection of key-value pairs.

```
const user = {  
  name: "Pavan",  
  age: 22,  
  isStudent: true  
};
```

## Creating Objects

### 1. Object Literal

- The simplest and most commonly used way.

```
const user = {  
  name: "Pavan",  
  age: 22  
};
```

### 2. Object Constructor

- Creates an empty object and adds properties later.

```
const user = new Object();
```

```
user.name = "Pavan";  
user.age = 22;
```

### 3. Constructor Function

```
function User(name, age) {  
    this.name = name;  
    this.age = age;  
}  
  
const user1 = new User("Pavan", 22);  
const user2 = new User("John", 25);
```

### 4. Class

```
class User {  
    constructor(name, age) {  
        this.name = name;  
        this.age = age;  
    }  
}  
  
const user = new User("Pavan", 22);
```

### 5. Factory Function

- A regular function that returns an object.

```
function createUser(name, age) {  
    return {  
        name,  
        age  
    };  
}  
  
const user = createUser("Pavan", 22);
```

### 6. Object.create()

- Creates a new object using another object as its prototype.

```
const person = {
  greet() {
    console.log("Hello");
  }
};

const user = Object.create(person);

user.name = "Pavan";
```

---

## Accessing Properties

### Dot Notation

```
console.log(user.name);
```

### Bracket Notation

```
console.log(user["name"]);
```

---

## this Keyword

- `this` is a keyword that refers to the object from which the current function is invoked. Its value is determined at runtime based on how the function is called.

### 1. `this` in an Object Method

- Inside an object method, `this` refers to **that object**.

```
const user = {
  name: "Pavan",

  greet() {
    console.log(this.name);
  }
};

user.greet(); // Pavan
```

```
const fn = user.greet; // the function is detached from the object, so
`this` is no longer `user`.
fn(); // empty string
```

## 2. `this` in a Regular Function

```
function greet() {
  console.log(this);
}

greet();// window in browser
```

## 3. `this` in an Arrow Function

- Arrow functions **do not have their own** `this` they inherit `this` from their surrounding (lexical) scope.

```
const user = {
  name: "Pavan",

  greet: () => {
    console.log(this);
  }
};

user.greet();
```

- `this` **does not refer to** `user` , It refers to whatever `this` was in the surrounding scope.

---

## `call()` , `apply()` , and `bind()`

- In JavaScript, `call()` , `apply()` , and `bind()` are built-in methods used to control the value of the `this` keyword inside a function.

### `call()`

- `call()` invokes a function immediately while explicitly setting the value of `this` .
- Arguments are passed individually.



```
functionName.call(thisArg, arg1, arg2, ...)
```

```
const user = {  
  name: "Pavan"  
};  
  
function greet(city) {  
  console.log(`Hello ${this.name} from ${city}`);  
}  
  
greet.call(user, "Bangalore");
```

## apply()

- apply() invokes a function immediately while explicitly setting the value of this.
- Arguments are passed as an array.

```
functionName.apply(thisArg, [arg1, arg2])
```

```
const user = {  
  name: "Pavan"  
};  
  
function greet(city, country) {  
  console.log(`Hello ${this.name} from ${city}, ${country}`);  
}  
  
greet.apply(user, ["Bangalore", "India"]);
```

## bind()

- bind() returns a new function with this permanently bound to the specified object.
- It does not invoke the function immediately.
- Use bind() when you need to **preserve the value of this** for later execution.

```
const newFunction = functionName.bind(thisArg);
```

```
const user = {  
  name: "Pavan"  
};
```

```
function greet() {  
    console.log(`Hello ${this.name}`);  
}  
  
const boundGreet = greet.bind(user);  
  
boundGreet();
```

---

## Object Static Methods

### Object.keys()

- `Object.keys()` is a static method that returns an array containing the enumerable property keys of an object.

```
const user = {  
    name: "Pavan",  
    age: 22,  
    city: "Bangalore"  
};  
  
const keys = Object.keys(user);  
  
console.log(keys);  
  
// ["name", "age", "city"]
```

### Object.values()

- `Object.values()` is a static method that returns an array containing the enumerable property values of an object.

```
const user = {  
    name: "Pavan",  
    age: 22,  
    city: "Bangalore"  
};  
  
console.log(Object.values(user));  
  
// ["Pavan", 22, "Bangalore"]
```

## Object.entries()

- `Object.entries()` is a static method that returns an array of an object's enumerable key-value pairs.

```
const user = {
  name: "Pavan",
  age: 22
};

console.log(Object.entries(user));

/*
[
  ["name", "Pavan"],
  ["age", 22]
]
*/
```

```
const user = {
  name: "Pavan",
  age: 22
};

for (const [key, value] of Object.entries(user)) {
  console.log(key, value);
}
```

## Object.freeze()

- `Object.freeze()` makes an object immutable. After freezing, properties cannot be added, removed, or modified.

```
const user = {
  name: "Pavan",
  age: 22
};

Object.freeze(user);

user.name = "John";    // Ignored
user.city = "Bangalore"; // Ignored
delete user.age;       // Ignored
```

```
console.log(user);
```

```
{
  name: "Pavan",
  age: 22
}
```

```
console.log(Object.isFrozen(user));
```

## Object.seal()

- `Object.seal()` prevents adding or deleting properties, but allows modifying existing property values.

```
const user = {
  name: "Pavan",
  age: 22
};

Object.seal(user);

user.name = "John";      // Allowed
user.city = "Bangalore"; // Ignored
delete user.age;         // Ignored

console.log(user);
```

```
{
  name: "John",
  age: 22
}
```

```
console.log(Object.isSealed(user));
```

Both `Object.freeze()` and `Object.seal()` are **shallow**.

```
const user = {
  name: "Pavan",
  address: {
    city: "Bangalore"
  }
}
```

```
    }  
};  
  
Object.freeze(user);  
  
user.address.city = "Mysore"; // Allowed  
  
console.log(user.address.city); // Mysore
```

- Because only the **top-level object** is frozen. The nested `address` object is still mutable.

---

## Arrays

- An array is a reference data type used to store an ordered collection of multiple values under a single variable. Each element is identified by its index, which starts from `0`.

### Creating Arrays

#### Method 1: Array Literal (Most Common)

```
const fruits = ["Apple", "Banana", "Orange"];  
console.log(fruits.length);
```

#### Method 2: Array Constructor

```
const fruits = new Array("Apple", "Banana", "Orange");
```

### 1. Adding / Removing Elements

| Method                 | Purpose                     |
|------------------------|-----------------------------|
| <code>push()</code>    | Add element(s) at end       |
| <code>pop()</code>     | Remove last element         |
| <code>shift()</code>   | Remove first element        |
| <code>unshift()</code> | Add element(s) at beginning |

```
const arr = [10, 20];  
  
arr.push(30); // [10, 20, 30]
```

```
arr.pop();           // [10, 20]

arr.unshift(5);      // [5, 10, 20]
arr.shift();          // [10, 20]
```

## 2. Searching

### includes()

- Returns `true` if the specified element exists in the array; otherwise returns `false`.

```
const nums = [10, 20, 30];

console.log(nums.includes(20)); // true
console.log(nums.includes(50)); // false
```

### indexOf()

### find()

- Returns the **first element** that satisfies the given condition.

```
const nums = [5, 10, 15, 20];

const result = nums.find(num => num > 10);

console.log(result); // 15
```

### findIndex()

- Returns the **index of the first element** that satisfies the given condition.

```
const nums = [5, 10, 15, 20];

const index = nums.findIndex(num => num > 10);

console.log(index); // 2
```

## 3. Transformation

### map()

- Creates a new array by applying a function to every element.

```
const nums = [1, 2, 3];

const doubled = nums.map(num => num * 2);

console.log(doubled); // [2, 4, 6]
```

## filter()

- Creates a new array containing only the elements that satisfy a condition.

```
const nums = [1, 2, 3, 4];

const even = nums.filter(num => num % 2 === 0);

console.log(even); // [2, 4]
```

## reduce()

- Reduces all elements of an array to a single value.

```
const nums = [1, 2, 3, 4];

const sum = nums.reduce((acc, curr) => acc + curr, 0);

console.log(sum);
```

## 4. Iteration

```
const nums = [1, 2, 3];

nums.forEach(num => {
  console.log(num);
});
```

## 5. Modification

### slice()

- Returns a shallow copy of a portion of an array **without modifying the original array**.

```
const nums = [10, 20, 30, 40];
```

```
const result = nums.slice(1, 3);

console.log(result); // [20, 30]
```

## splice()

- Adds, removes, or replaces elements **by modifying the original array**.

```
nums.splice(startIndex,NumberOfElementsToRemove,..New Elements)
```

### Remove

```
const nums = [10, 20, 30, 40];

nums.splice(1, 2);

console.log(nums); // [10, 40]
```

### Insert

```
const nums = [10, 40];

nums.splice(1, 0, 20, 30);

console.log(nums); // [10, 20, 30, 40]
```

### Replace

```
const nums = [10, 20, 30];

nums.splice(1, 1, 100);

console.log(nums); // [10, 100, 30]
```

## sort()

- Sorts the elements of an array **in place**.

```
const fruits = ["Banana", "Apple", "Orange"];

fruits.sort();
```



```
console.log(fruits);
```

## reverse()

- `reverse()` reverses the order of the elements in an array by modifying the original array.

```
const nums = [1, 2, 3, 4];  
  
nums.reverse();  
  
console.log(nums); // [4, 3, 2, 1]
```

## Validation

### some()

- `some()` returns `true` if at least one element satisfies the given condition; otherwise it returns `false`.

```
const nums = [1, 3, 5, 8];  
  
const result = nums.some(num => num % 2 === 0);  
  
console.log(result); // true
```

### every

- `every()` returns `true` only if all elements satisfy the given condition.

```
const nums = [2, 4, 6];  
  
const result = nums.every(num => num % 2 === 0);  
  
console.log(result); // true
```

## Combining & Flattening

### concat()

- `concat()` returns a new array by combining two or more arrays without modifying the original arrays.

```
const arr1 = [1, 2];
const arr2 = [3, 4];

const result = arr1.concat(arr2);

console.log(result); // [1, 2, 3, 4]
```

## flat()

- `flat()` returns a new array by flattening nested arrays up to the specified depth.

```
const arr = [1, 2, [3, 4]];

console.log(arr.flat()); // [1, 2, 3, 4]
```

---

# Destructuring

- Destructuring is an ES6 feature that allows you to extract values from arrays or properties from objects into separate variables using a concise syntax.

## Types of Destructuring

### 1. Object Destructuring

- Extracts properties from an object.

```
const user = {
  name: "Pavan",
  age: 22
};

const { name, age } = user;

console.log(name); // Pavan
console.log(age); // 22
```

### 2. Array Destructuring

- Extracts values based on their position (index).

```
const colors = ["Red", "Green", "Blue"];

const [first, second] = colors;

console.log(first);
console.log(second);
```

## Skipping Elements

```
const colors = ["Red", "Green", "Blue"];

const [first, , third] = colors;

console.log(first);
console.log(third);
```

## Default Values

- Used when the property or element doesn't exist.

```
const user = {
  name: "Pavan"
};

const { name, age = 22 } = user;

console.log(age);
```

## Renaming Variables

```
const user = {
  name: "Pavan",
  age: 22
};

const { name: fullName } = user;

console.log(fullName);
```

## Nested Destructuring

```
const user = {
  name: "Pavan",
  address: {
    city: "Bangalore",
    state: "Karnataka"
  }
};

const {
  address: { city }
} = user;

console.log(city); // Bangalore
```

---

## Spread Operator ( ... )

- The spread operator ( ... ) expands an iterable or object into its individual elements or properties.
- Used for copying, merging.

```
const arr1 = [1, 2];
const arr2 = [3, 4];

const arr3 = [ ... arr1, ... arr2];

console.log(arr3);
```

```
const user = {
  name: "Pavan",
  age: 22
};

// Spread Creates a Shallow Copy
const copy = {
  ... user
};

console.log(copy);
```

---

# Rest Operator ( ... )

- The Rest Operator ( ... ) collects multiple values into a single array or object.
- The Rest Operator is commonly used with:
  - Function Parameters
  - Array Destructuring
  - Object Destructuring

```
function sum( ... nums ) {  
    console.log( nums );  
}  
  
sum( 1, 2, 3 );
```

## Rest in Array Destructuring

```
const nums = [10, 20, 30, 40];  
  
const [first, ... remaining] = nums;  
  
console.log( first );  
console.log( remaining );
```

## Rest in Object Destructuring

```
const user = {  
    name: "Pavan",  
    age: 22,  
    city: "Bangalore"  
};  
  
const { name, ... details } = user;  
  
console.log( name );  
console.log( details );
```

| Rest Parameters          | arguments Object                       |
|--------------------------|----------------------------------------|
| Real Array               | Array-like Object                      |
| Supports array methods   | Doesn't directly support array methods |
| Works in arrow functions | Not available in arrow functions       |

| Rest Parameters | arguments Object         |
|-----------------|--------------------------|
| ES6 feature     | Older JavaScript feature |

| Spread                             | Rest                                       |
|------------------------------------|--------------------------------------------|
| Expands values                     | Collects values                            |
| Used while passing or copying data | Used while receiving or destructuring data |
| Produces individual values         | Produces an array or object                |

---

## Template Literals

- Template literals are a feature introduced in ES6 that allow embedding expressions, creating multi-line strings, and formatting strings more easily using backticks ( ``` ).

```
const name = "Pavan";
const age = 22;

console.log(`My name is ${name} and I am ${age} years old.`);
```

```
const text = `Hello
Welcome
JavaScript`;

console.log(text);
```

```
Hello
Welcome
JavaScript
```

## String Interpolation

- String interpolation is the process of embedding variables or expressions inside a string using `${}` .

```
const name = "Pavan";

console.log(`Hello ${name}`);
```

---

# Enhanced Object Literals

- Enhanced Object Literals are ES6 features that provide a shorter and more convenient syntax for creating and defining objects.
- They include:
  - Property Shorthand
  - Method Shorthand
  - Computed Property Names

## 1. Property Shorthand

- Property shorthand allows you to create object properties using variable names without repeating the property name and variable name.

```
const name = "Pavan";
const age = 22;

const user = {
  name,
  age
};

console.log(user);
```

## 2. Method Shorthand

- Method shorthand allows defining object methods without using the `function` keyword.

```
const user = {
  greet() {
    console.log("Hello");
  }
};
```

## 3. Computed Property Names

- Computed property names allow you to create object property names dynamically using expressions inside square brackets ( `[]` ).

```
const key = "name";
```

```
const user = {  
  [key]: "Pavan"  
};  
  
console.log(user);
```

```
const prefix = "user";  
  
const obj = {  
  [`${prefix}Id`]: 101  
};  
  
console.log(obj);
```

---

## Modules

- A module is a JavaScript file that encapsulates related code and can export or import variables, functions, classes, or objects from other modules.
- The `export` keyword is used to make variables, functions, classes, or objects available for use in other modules.
- There are two types of exports:
  - Named Export
  - Default Export

### Named Export

math.js

```
export const PI = 3.14;  
  
export function add(a, b) {  
  return a + b;  
}  
  
export function subtract(a, b) {  
  return a - b;  
}
```

- Multiple named exports are allowed.

app.js



```
import { PI, add, subtract } from "./math.js";

console.log(PI);
console.log(add(10, 20));
```

## Import Aliasing

```
import { add as sum } from "./math.js";

console.log(sum(10, 20));
```

## Default export

```
export default function add(a, b) {
  return a + b;
}
```

```
import add from "./math.js";

console.log(add(10, 20));
```

## ES Modules

- **ES Modules** are the **standard JavaScript module system** introduced in **ES6**

```
// math.js

export function add(a, b) {
  return a + b;
}
```

```
// app.js

import { add } from "./math.js";

console.log(add(10, 20));
```

## CommonJS Modules

- **CommonJS** is the traditional module system widely used in Node.js.

```
// math.js

function add(a, b) {
  return a + b;
}

module.exports = { add };
```

```
// app.js

const { add } = require("./math");

console.log(add(10, 20));
```

---

## Execution Context

- An Execution Context is an abstract environment created by the JavaScript engine to evaluate and execute JavaScript code.
- It contains everything needed to execute the code, including variables, functions, and the value of `this`.
- Whenever JavaScript executes a piece of code, it first creates an Execution Context for that code.

## Types of Execution Context

### 1. Global Execution Context (GEC)

- JavaScript first creates the **Global Execution Context**.
- Every program has exactly **one** Global Execution Context.

```
let a = 10;

console.log(a);
```

### 2. Function Execution Context (FEC)

- Created every time a function is invoked.

```
function greet() {  
    console.log("Hello");  
}  
  
greet();
```

## Components of an Execution Context

- An execution context consists of three main components.

```
Execution Context  
|  
├── Variable Environment  
├── Lexical Environment  
└── this Binding
```

### 1. Variable Environment

- Stores variables & function declaration

### 2. Lexical Environment

- Stores information about current scope & parent scope

### 3. `this` Binding

- Stores the value of `this`

## Life Cycle of an Execution Context

### Phase 1: Memory Creation Phase

- JavaScript scans the entire code and allocates the memory for variables & function.

### Phase 2: Execution Phase

- JavaScript executes code line by line.

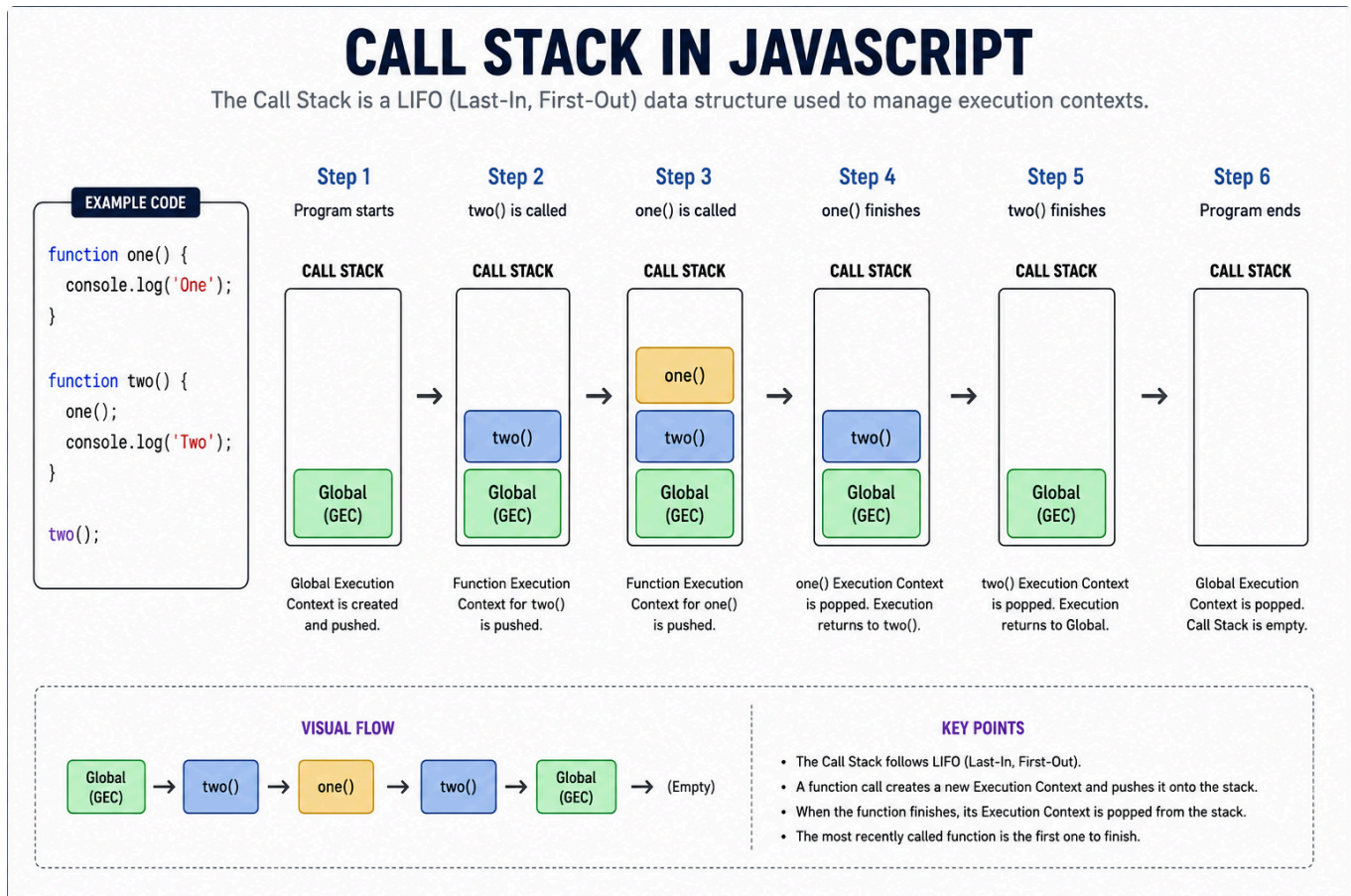
---

## Call Stack

- The Call Stack is a Last-In, First-Out (LIFO) data structure used by the JavaScript engine to manage and execute execution contexts.

## What causes a Stack Overflow?

- A Stack Overflow occurs when execution contexts continue to be pushed onto the Call Stack without being removed, typically due to infinite or uncontrolled recursion.



## Lexical Environment

- A Lexical Environment is an internal data structure that stores variables, functions, and a reference to the parent lexical environment, allowing JavaScript to resolve identifiers based on lexical scope.

## What are the components of a Lexical Environment?

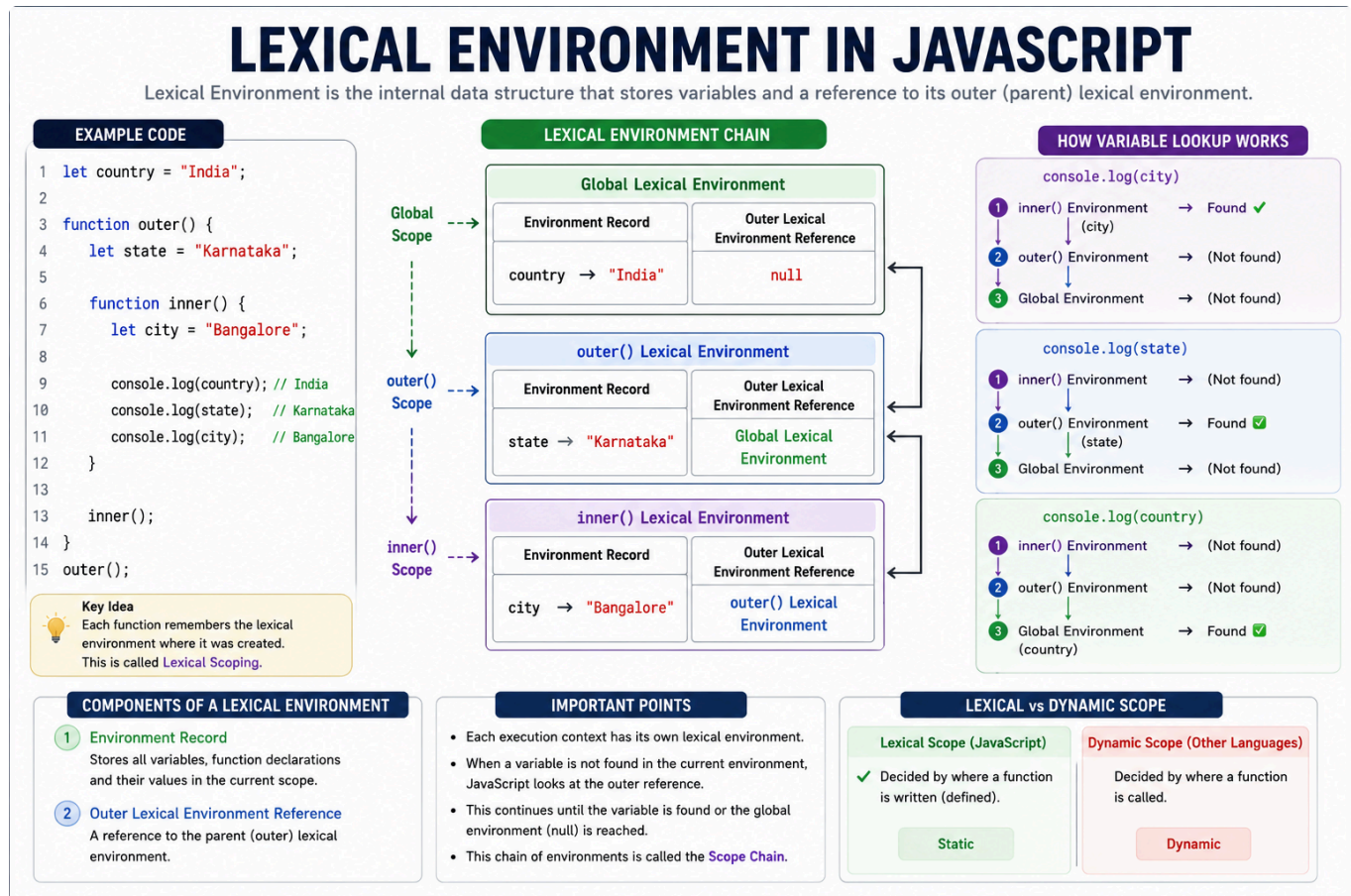
- Environment Record
- Outer Lexical Environment Reference

## What is stored in the Environment Record?

- Variables
- Function declarations

# Scope Chain

- The Scope Chain is the mechanism JavaScript uses to resolve variables by searching the current scope first and then moving outward through its parent lexical scopes until the variable is found or the global scope is reached.



## Asynchronous JavaScript

### Synchronous JavaScript

- Synchronous execution means JavaScript executes one statement at a time in the order they appear. Each statement must complete before the next one begins.

```
console.log("Start");
```

```
console.log("Middle");
```

```
console.log("End");
```

```
Start
Middle
End
```

## Asynchronous JavaScript

- Asynchronous execution allows certain operations to start without blocking the execution of the remaining code. The result is handled later when the operation completes.

```
console.log("Start");

setTimeout(() => {
    console.log("Inside Timer");
}, 2000);

console.log("End");
```

```
Start
End
Inside Timer
```

## Blocking

- A task prevents other tasks from executing until it completes.

```
console.log("Start");

// Heavy computation

for (let i = 0; i < 10000000000; i++) {}

console.log("End");
```

## Non-Blocking

- The current operation doesn't stop JavaScript from executing the remaining code.

```
console.log("Start");

setTimeout(() => {
    console.log("Done");
}, 2000);
```

```
console.log("End");
```

## How javascript handles multiple tasks if its single threaded

- JavaScript executes on a **single thread**.
- The **browser or Node.js runtime** provides asynchronous APIs.
- Long-running operations (timers, network requests, file I/O, etc.) are handled by the runtime, not by the JavaScript engine.
- When those operations finish, their callbacks are placed into queues.
- The **Event Loop** decides when those callbacks can move to the Call Stack.

## Is JavaScript synchronous or asynchronous?

- JavaScript executes code synchronously by default.
- It supports asynchronous operations through runtime-provided APIs (Browser APIs or Node.js APIs).

## Browser Runtime

- A Browser Runtime is the environment that executes JavaScript in the browser.
- It consists of the JavaScript engine (V8), Web APIs, Callback Queues, and the Event Loop.

```
Browser Runtime
|
├── JavaScript Engine (V8)
├── Web APIs
├── Callback Queue
├── Microtask Queue
└── Event Loop
```

## Web APIs

- Web APIs are browser-provided functionalities that allow JavaScript to perform operations outside the capabilities of the JavaScript engine.

| Web API                    | Purpose                              |
|----------------------------|--------------------------------------|
| <code>setTimeout()</code>  | Execute code after a delay           |
| <code>setInterval()</code> | Execute repeatedly after an interval |
| <code>fetch()</code>       | Make HTTP requests                   |

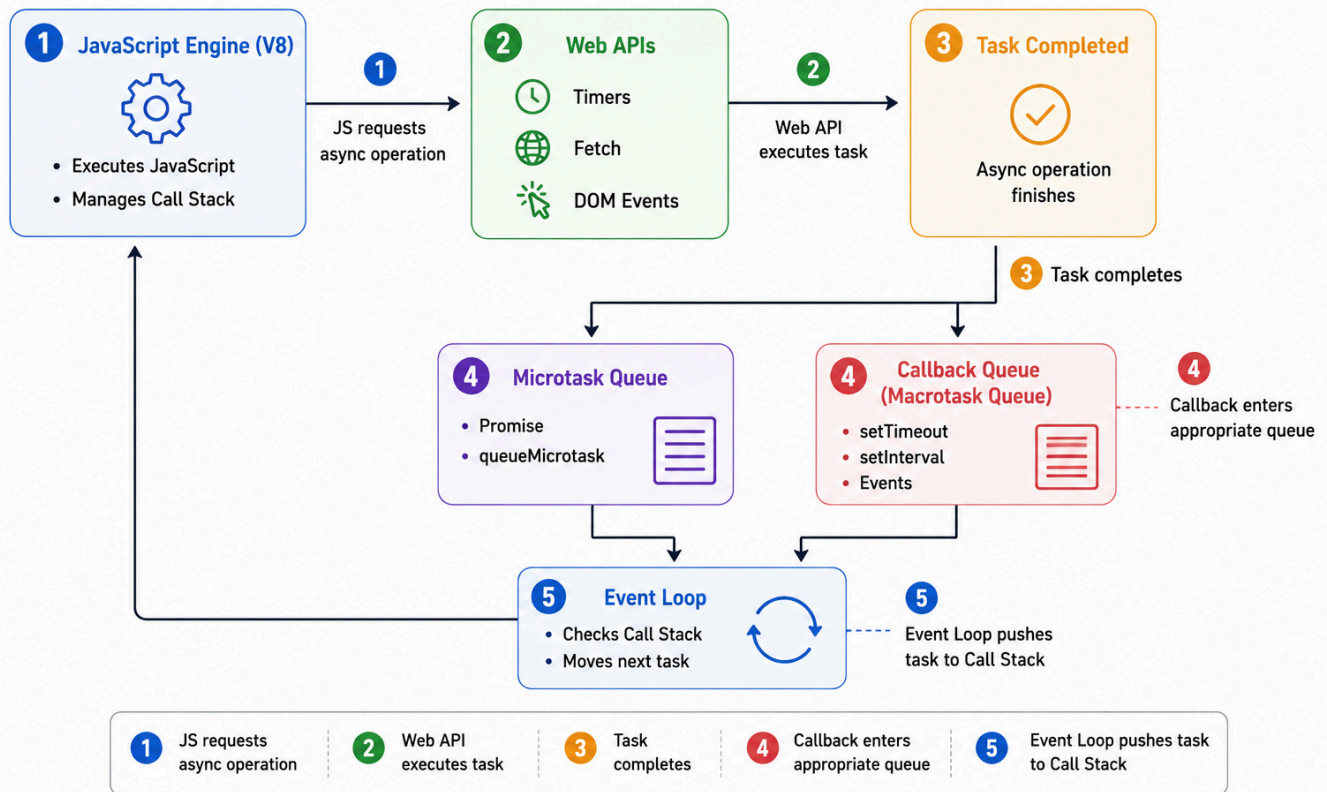
| Web API         | Purpose                   |
|-----------------|---------------------------|
| DOM API         | Access and modify HTML    |
| Event Listeners | Handle user interactions  |
| Local Storage   | Store data in the browser |
| Geolocation     | Access device location    |
| WebSocket       | Real-time communication   |

| Browser Runtime | Node.js Runtime             |
|-----------------|-----------------------------|
| Web APIs        | Node APIs                   |
| DOM             | No DOM                      |
| Fetch API       | File System, HTTP, Streams  |
| Event Listeners | File Events, Network Events |
| Runs in browser | Runs outside browser        |

| JavaScript Engine      | Runtime                            |
|------------------------|------------------------------------|
| Executes JavaScript    | Provides additional APIs           |
| Manages Call Stack     | Manages asynchronous operations    |
| Parses & compiles code | Handles timers, networking, events |
| Example: V8            | Browser, Node.js                   |



# BROWSER RUNTIME



## 1. JavaScript Engine (V8)

- Executes JavaScript code on the **Call Stack**.
- When it encounters an asynchronous operation such as a timer, `fetch()`, or a DOM event, it delegates that operation to the Browser Web API.
- V8 continues executing the remaining synchronous JavaScript instead of waiting.

## 2. Web APIs

- The browser handles the actual asynchronous operation.
- For example, a timer is managed by the browser, and a network request is handled by the browser's networking capabilities.
- The Web API does not execute the JavaScript callback.
- It only performs the asynchronous operation and determines when it has completed.

## 3. Async Operation Completes

- Once the operation finishes, the result, error, or event information becomes available.
- The associated JavaScript callback is now ready to be executed.
- The callback itself has **not been executed yet**.

## 4. Callback goes to a Queue

- The completed asynchronous operation causes the associated JavaScript work to be scheduled.
- Promise reactions go to the **Microtask Queue**.

- Timer and DOM event callbacks generally go to the **Callback (Macrotask) Queue**.
- The queue temporarily holds the JavaScript callback until V8 is ready to execute it.

## 5. Event Loop

- The Event Loop continuously checks whether the **Call Stack is empty**.
- When JavaScript is ready to execute another task, the Event Loop takes the appropriate queued callback and places it on the Call Stack.
- The Event Loop does **not execute the callback itself**. It only coordinates when the callback can be moved to the Call Stack.

## 6. JavaScript Engine (V8) executes the callback

- The callback is now back on the **Call Stack**.
- V8 executes the callback as normal JavaScript.
- The callback can use the result, error, or event data produced by the asynchronous operation.

---

# Callbacks

- A callback is a function passed as an argument to another function, which is then called later when a particular operation is completed or an event occurs.
- Callbacks are commonly used to execute JavaScript code after an asynchronous operation finishes.

```
setTimeout(() => {  
    console.log("Timer finished");  
}, 2000);
```

## Callback Hell

- Callback Hell is a situation where multiple nested callbacks make asynchronous code deeply nested, difficult to read, maintain, and handle errors in.  
callbacks

```
getUser(function(user) {  
  
    getOrders(user, function(orders) {  
  
        getOrderDetails(orders, function(order) {  
  
            processPayment(order, function(payment) {
```

```

        sendEmail(payment, function(result) {

            console.log("Completed");

        });

    });

});

});

});

```

## Problems with Callback Hell

- Code becomes deeply nested.
- Difficult to read.
- Difficult to maintain.
- Error handling becomes complicated.
- Difficult to modify the execution flow.
- Reusing callbacks becomes harder.

## Why Promises Were Introduced

- Callbacks work, but deeply nested asynchronous operations become difficult to manage.
- Promises provide a cleaner way to represent asynchronous results.

promises

```

getUser()
    .then(user => getOrders(user))
    .then(orders => getOrderDetails(orders))
    .then(order => processPayment(order))
    .then(result => console.log(result))
    .catch(error => console.log(error));

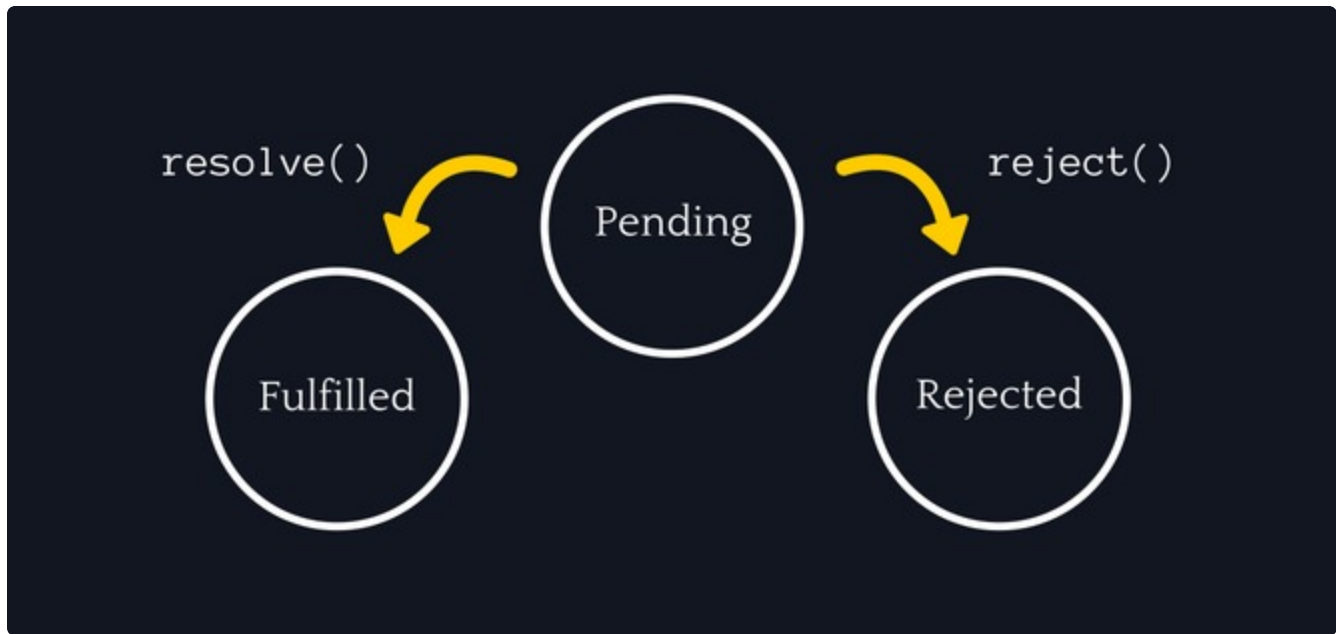
```

| Callback                            | Promise                           |
|-------------------------------------|-----------------------------------|
| Function passed to another function | Object representing future result |
| Can lead to deep nesting            | Supports chaining                 |
| Error handling can become difficult | Centralized <code>.catch()</code> |

| Callback                    | Promise                   |
|-----------------------------|---------------------------|
| Common in older async APIs  | Modern async programming  |
| Can result in Callback Hell | Helps avoid Callback Hell |

## Promises

- A Promise is an object that represents the eventual completion or failure of an asynchronous operation and its resulting value.



### Creating Promise using the `Promise` constructor.

```
const promise = new Promise((resolve, reject) => {  
    // asynchronous operation  
});
```

- `resolve()` is used when the operation succeeds.
- `reject()` is used when the operation fails.

### Consuming a Promise

- We usually consume a Promise using `.then()` and `.catch()`.
- `.then()` executes when the Promise is fulfilled.
- `.catch()` handles Promise rejection.
- `.finally()` executes regardless of whether the Promise is fulfilled or rejected.

```
const promise = new Promise((resolve, reject) => {
  resolve("Success");
});

promise
  .then(result => {
    console.log(result);
  })
  .catch(error => {
    console.log(error);
  });
  .finally(() => {
    console.log("Operation completed");
  });
```

## Promise Chaining

```
getUser()
  .then(user => {
    return getOrders(user);
  })
  .then(orders => {
    return getOrderDetails(orders);
  })
  .then(details => {
    console.log(details);
  })
  .catch(error => {
    console.log(error);
  });
```

- Each `.then()` can return another Promise.
- The next `.then()` waits for that Promise.

## Promise Utility Methods

- The four important methods are:
  - `Promise.all()`
  - `Promise.allSettled()`
  - `Promise.race()`
  - `Promise.any()`

### 1. `Promise.all()`

- `Promise.all()` waits for multiple Promises and fulfills only when **all Promises fulfill**.

```
const p1 = Promise.resolve("User");
const p2 = Promise.resolve("Orders");
const p3 = Promise.resolve("Payment");

Promise.all([p1, p2, p3])
  .then(results => {
    console.log(results);
  });
```

```
["User", "Orders", "Payment"]
```

- The results are returned in the **same order as the input Promises**.
- `Promise.all()` rejects as soon as one of the Promises rejects.

## 2. Promise.allSettled()

- `Promise.allSettled()` waits for **every Promise to finish**, regardless of whether it fulfills or rejects.

```
const p1 = Promise.resolve("User");
const p2 = Promise.reject("Orders failed");
const p3 = Promise.resolve("Payment");

Promise.allSettled([p1, p2, p3])
  .then(results => {
    console.log(results);
  });
```

```
[
  { status: "fulfilled", value: "User" },
  { status: "rejected", reason: "Orders failed" },
  { status: "fulfilled", value: "Payment" }
]
```

## 3. Promise.race()

- `Promise.race()` settles as soon as the **first Promise settles**, whether it fulfills or rejects.

```
const p1 = new Promise(resolve => {
  setTimeout(() => resolve("First"), 1000);
});
```

```
});

const p2 = new Promise(resolve => {
  setTimeout(() => resolve("Second"), 2000);
});

Promise.race([p1, p2])
  .then(result => {
    console.log(result);
  });
```

First

## 4. Promise.any()

- `Promise.any()` fulfills as soon as the **first Promise fulfills** and ignores rejected Promises unless all Promises reject.

```
const p1 = new Promise((resolve, reject) => {
  setTimeout(() => reject("Failed"), 1000);
});

const p2 = new Promise(resolve => {
  setTimeout(() => resolve("Success"), 2000);
});

Promise.any([p1, p2])
  .then(result => {
    console.log(result);
  });
```

Success

---

## Async/Await

- `async/await` is syntactic sugar over Promises that allows asynchronous code to be written in a synchronous-looking manner.

```
async function getData() {
  const result = await Promise.resolve("Success");
}
```

```
    console.log(result);  
}  
  
getData();
```

## Error Handling

With Promise chaining:

```
fetch("/users")  
  .then(response => response.json())  
  .catch(error => {  
    console.log(error);  
  });
```

With async/await

```
async function getUsers() {  
  try {  
    const response = await fetch("/users");  
  
    const data = await response.json();  
  
    console.log(data);  
  } catch (error) {  
    console.log(error);  
  }  
}
```

## Running Promises in Parallel

- If the operations don't depend on each other, don't unnecessarily await them one by one.

```
async function getData() {  
  const [users, products] = await Promise.all([  
    getUsers(),  
    getProducts()  
  ]);  
  
  console.log(users);  
  console.log(products);  
}
```



- `await` pauses execution of the current `async` function.
- 

## Error Handling

- JavaScript provides error-handling mechanisms so your program can handle unexpected situations without crashing the entire execution flow.
- The `try` block contains code that might throw an error.
- The `catch` block handles an error thrown from the corresponding `try` block.
- The `error` object contains information about the error.
- The `throw` statement is used to explicitly generate an error.
- The `finally` block executes after `try / catch`, regardless of whether an error occurred.

```
function getUser(shouldSucceed) {
  return new Promise((resolve, reject) => {

    setTimeout(() => {
      if (shouldSucceed) {
        resolve({ id: 101, name: "Pavan" });
      } else {
        reject(new Error("Failed to fetch user"));
      }
    }, 1000);

  });
}

async function fetchUser() {
  try {
    const user = await getUser(true);

    console.log("User:", user);

  } catch (error) {
    console.log("Error:", error.message);

  } finally {
    console.log("Request completed");
  }
}

fetchUser();
```

## Error Object

- When JavaScript encounters or creates an error, it represents that error using an **Error object**

```
const error = new Error("Something went wrong");

console.log(error.name);    // Error
console.log(error.message); // Something went wrong
console.log(error.stack);   // Stack trace
```

| Property | Meaning                                     |
|----------|---------------------------------------------|
| name     | Type/name of the error                      |
| message  | Description of the error                    |
| stack    | Where the error occurred and the call stack |

## Stack

```
Error: Something went wrong
    at getUser (...)
    at main (...)
```

## Custom Errors

- Custom errors allow you to create your own error types for specific situations in your application.

```
class InsufficientBalanceError extends Error {
  constructor(message) {
    super(message);
    this.name = "InsufficientBalanceError";
  }
}
```

```
function withdraw(balance, amount) {
  if (amount > balance) {
    throw new InsufficientBalanceError("Insufficient balance");
  }
}
```

```
    return balance - amount;
}
```

---

## Prototypes & OOP

### Prototype

- Every JavaScript object has an internal link to another object called its **prototype**.
- This allows objects to inherit properties and methods from other objects.

```
const user = {
  name: "Pavan"
};

console.log(Object.getPrototypeOf(user));
```

```
[Object: null prototype] {}
```

```
const user = {
  name: "Pavan"
};

console.log(user.toString()); // [object Object]

console.log(user.hasOwnProperty("toString")); // false

console.log(Object.getPrototypeOf(user).hasOwnProperty("toString")); // true
```

---

### Prototype Chain

- The **Prototype Chain** is the mechanism JavaScript uses to find a property or method when it is not present directly on an object.
  - If a property or method is not found on an object, JavaScript searches its `[[Prototype]]`, then that prototype's `[[Prototype]]`, and continues until it finds the property or reaches `null`.
-

# Classes

```
class User {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  sayHello() {
    console.log(`Hello ${this.name}`);
  }
}

const user = new User("Pavan", 25);

user.sayHello();
```

## Class Inheritance

```
class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log(`${this.name} makes a sound`);
  }
}

class Dog extends Animal {
  constructor(name, breed) {
    super(name);
    this.breed = breed;
  }

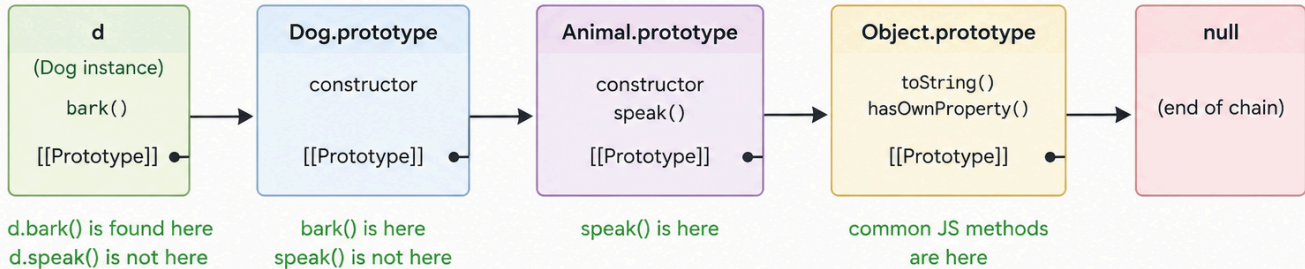
  bark() {
    console.log(`${this.name} barks`);
  }
}

const dog = new Dog("Bruno", "Labrador");

dog.speak();
dog.bark();
```

## JavaScript OOP uses Prototype Chain (Even with Classes)

```
class Animal {  
  speak() { console.log("Animal speaks"); }  
}  
  
class Dog extends Animal {  
  bark() { console.log("Dog barks"); }  
}  
  
const d = new Dog();
```



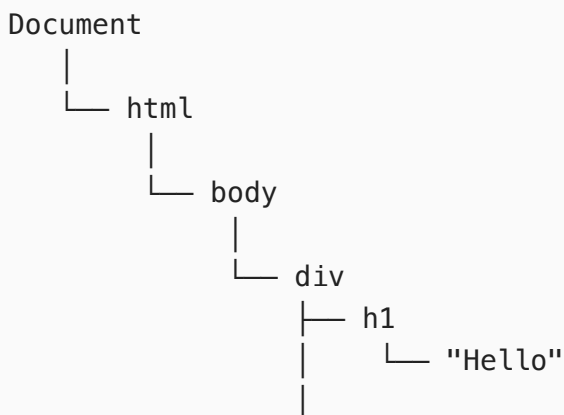
### How it works:

- JS looks for a property/method on the object itself.
- If not found, it goes to `[[Prototype]]` and keeps going up the chain.
- This is Prototype Chaining.

## DOM (Document Object Model)

- DOM is a interface provided by the browser that represents an HTML document as a tree of objects, allowing JavaScript to read and modify the page.

```
<div>  
  <h1>Hello</h1>  
  <p>Welcome</p>  
</div>
```



```
└─ p
  └─ "Welcome"
```

- Each HTML element becomes a **DOM node/object**.

```
const heading = document.querySelector("h1");

heading.textContent = "Hello Pavan";
```

## Methods to interact with DOM

### Selecting Elements

#### `getElementById`

- Selects an element by its `id`.

```
const heading = document.getElementById("title");
```

#### `querySelector()`

- Returns the first matching element.

```
const heading = document.querySelector("h1");
```

#### `querySelectorAll()`

- Returns all matching elements.

```
const items = document.querySelectorAll("li");
```

### Creating Elements

- JavaScript can also create new DOM elements:

```
const paragraph = document.createElement("p");

paragraph.textContent = "Hello Pavan";

document.body.appendChild(paragraph);
```

## Modifying Elements

```
const image = document.querySelector("img");  
  
image.src = "photo.jpg";
```

```
const heading = document.querySelector("h1");  
  
heading.textContent = "Hello";  
heading.style.color = "red";
```

## Remove Elements

---

## DOM Events

- An **event** is an action or occurrence detected by the browser.
- Examples:
  - Mouse click
  - Keyboard press
  - Form submission
  - Mouse movement
  - Page load
- JavaScript can listen for these events and execute a function when they occur.

```
element.addEventListener("event", callback);
```

```
const button = document.querySelector("button");  
  
button.addEventListener("click", () => {  
    console.log("Button clicked");  
});
```

| Event     | When it occurs            |
|-----------|---------------------------|
| click     | Element is clicked        |
| dblclick  | Element is double-clicked |
| mouseover | Mouse moves over element  |

| Event    | When it occurs                 |
|----------|--------------------------------|
| mouseout | Mouse leaves element           |
| keydown  | Keyboard key is pressed        |
| keyup    | Keyboard key is released       |
| submit   | Form is submitted              |
| change   | Input value changes            |
| input    | Input value is being changed   |
| load     | Resource/page finishes loading |

## Event Object

- The browser passes an **event object** to the callback.

```
const button = document.querySelector("button");

button.addEventListener("click", (event) => {
  console.log(event);
});
```

- `event.target` tells you which element triggered the event.

### preventDefault()

- Prevents the default behaviour of an event for customizing purpose

```
form.addEventListener("submit", (event) => {
  event.preventDefault();

  console.log("Form submission handled by JavaScript");
});
```

### removeEventListener()

- You can remove an event listener, but you need the same function reference.

```
function handleClick() {
  console.log("Clicked");
}

button.addEventListener("click", handleClick);
```



```
button.removeEventListener("click", handleClick);
```

## Event Bubbling

- When an event happens on a child element, the event **bubbles upward** through its parent elements.
- Event listeners are **registered for the bubbling phase by default**.

product list with 100 products.

```
<ul id="productList">
  <li class="product">Product 1</li>
  <li class="product">Product 2</li>
  <li class="product">Product 3</li>
  <!-- ... -->
  <li class="product">Product 100</li>
</ul>
```

```
const productList = document.querySelector("#productList");

productList.addEventListener("click", (event) => {
  if (event.target.classList.contains("product")) {
    console.log("Selected product:", event.target.textContent);
  }
});
```

- The event starts at the element that was clicked and **bubbles upward**.

## Event Capturing

- The event travels from the top of the DOM toward the target element.

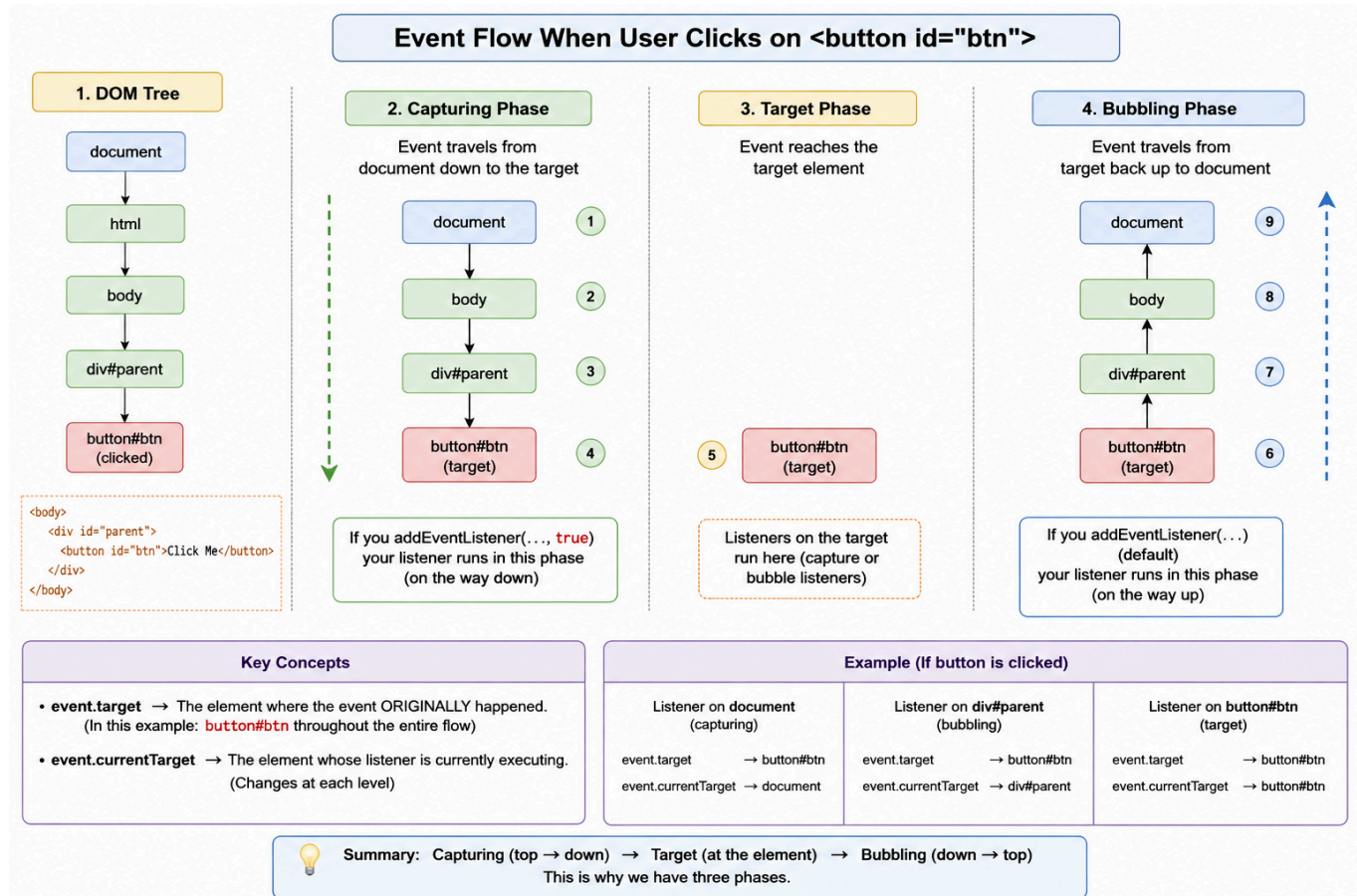
```
parent.addEventListener("click", () => {
  console.log("Parent");
}, true);
```

- You enable capturing using the third argument:
- `true` means the listener runs during the **capturing phase**.

# Event Phases

- An event generally passes through three phases:

1. Capturing
2. Target
3. Bubbling



## Event Delegation

- Instead of attaching an event listener to every child element, attach one listener to their common parent and use event bubbling to handle the events.

```
<ul id="list">  
  <li>Apple</li>  
  <li>Banana</li>  
  <li>Orange</li>  
</ul>
```

```
const list = document.querySelector("#list");

list.addEventListener("click", (event) => {
  if (event.target.tagName === "LI") {
    console.log("Clicked:", event.target.textContent);
  }
});
```

## event.target VS event.currentTarget

```
list.addEventListener("click", (event) => {
  console.log(event.target);
  console.log(event.currentTarget);
});
```

- `event.target` The element where the event **actually originated**.
- `event.currentTarget` The element whose **event listener is currently executing**.

```
event.target  
→ <li>
```

```
event.currentTarget  
→ <ul>
```

## Stop Bubbling - `stopPropagation`

```
child.addEventListener("click", (event) => {
  event.stopPropagation();

  console.log("Button clicked");
});
```

---

## Debouncing

- Debouncing delays function execution until a certain amount of time has passed since the last event.
- Example: Search box.

```
function debounce(callback, delay) {
  let timer;

  return function (...args) {
    clearTimeout(timer);

    timer = setTimeout(() => {
      callback(...args);
    }, delay);
  };
}
```

```
const search = debounce((query) => {
  console.log("API call:", query);
}, 500);

search("P");
search("Pa");
search("Pav");
search("Pava");
search("Pavan");
```

## Throttling

- Throttling limits a function so it executes at most once within a specified time interval.
- Example: Scroll event.

```
function throttle(callback, delay) {
  let lastTime = 0;

  return function (...args) {
    const currentTime = Date.now();

    if (currentTime - lastTime >= delay) {
      lastTime = currentTime;
      callback(...args);
    }
  };
}
```

```
const handleScroll = throttle(() => {
  console.log("Scroll handled");
}, 100);
```

```
window.addEventListener("scroll", handleScroll);
```

---

# Memory

## Shallow Copy

- A **shallow copy** creates a new object, but nested objects are still shared.

```
const user1 = {
  name: "Pavan",
  address: {
    city: "Bangalore"
  }
};

const user2 = { ...user1 };

console.log(user1 === user2);           // false
console.log(user1.address === user2.address); // true
```

## Deep Copy

- A **deep copy** creates a completely independent copy, including nested objects.
- `structuredClone()` is a built-in JavaScript API for creating a **deep copy** of supported JavaScript values.

```
const user1 = {
  name: "Pavan",
  address: {
    city: "Bangalore"
  }
};

const user2 = structuredClone(user1);

console.log(user1 === user2);           // false
console.log(user1.address === user2.address); // false
```

---

# Polyfill

- Polyfill is code that implements a JavaScript feature when the current environment doesn't natively support it.
- Suppose an old browser doesn't support: `Array.prototype.includes()`

You can provide your own implementation:

```
if (!Array.prototype.includes) {  
  Array.prototype.includes = function (value) {  
    return this.indexOf(value) !== -1;  
  };  
}
```

---

## Why is `let / const` preferred over `var` in modern JS?

- Block scoping avoids variables leaking out of `if / for` blocks.
- TDZ turns use-before-declaration into an immediate error instead of a silent `undefined`.
- No silent redeclaration bugs, no accidental global pollution.

## Classic closure-in-loop pitfall — what does this print?

```
for (var i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 0);  
}  
// 3 3 3 — all three callbacks share the SAME function-scoped `i`,  
// which equals 3 by the time any callback runs (loop finishes first)  
  
for (let j = 0; j < 3; j++) {  
  setTimeout(() => console.log(j), 0);  
}  
// 0 1 2 — `let` creates a fresh binding of `j` for every iteration
```

- `var` is **function-scoped**, so there is only **one** `i` **variable** shared by all three callbacks.
- `let` creates a **new binding for each loop iteration**.

## What happens to `this` when a method is extracted and called standalone?

```
const obj = { value: 42, getValue() { return this.value; } };  
const fn = obj.getValue;
```

```
console.log(fn()); // undefined – this is no longer bound to obj
```

- this depends on how a function is called. When a method is extracted and called standalone, this is no longer bound to the original object.

```
const fn = obj.getValue.bind(obj);  
  
console.log(fn()); // 42
```

## Why doesn't `setTimeout(fn, 0)` run immediately?

- `setTimeout()` is a **Browser Web API**, so JavaScript delegates the timer to the browser instead of waiting for it.
- Once the timer finishes, the browser puts the callback into the **Macrotask (Callback) Queue**.
- The **Event Loop** waits for the Call Stack to become empty and microtasks to finish, then moves the callback to the Call Stack for V8 to execute.

```
console.log("1");  
  
setTimeout(() => {  
  console.log("2");  
}, 0);  
  
console.log("3");
```

```
1  
3  
2
```

## \_\_proto\_\_ vs prototype — difference?

| prototype                                                                            | __proto__ ( [[Prototype]] )                                                                        |
|--------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| A property that exists only on <b>functions</b> (specifically constructor functions) | An accessor property that exists on <b>every object</b> , pointing to what it inherits from        |
| Defines what instances created via <code>new Fn()</code> will inherit                | The actual link walked during property lookup at runtime                                           |
| Accessed as <code>Fn.prototype</code>                                                | Accessed as <code>instance.__proto__</code> , and <code>instance.__proto__ === Fn.prototype</code> |

## Is `class` "real" classes in JavaScript?

- No — `class` is syntactic sugar over the existing prototype-based inheritance model. Methods defined in a class body get added to `ClassName.prototype`, and `extends` wires up the prototype chain — exactly what manual prototype code would do.

## ?? vs || — difference?

- `||` returns the right operand if the left is any **falsy** value (`0`, `''`, `NaN`, `false`, `null`, `undefined`).
- `??` (nullish coalescing) returns the right operand only if the left is specifically `null` or `undefined` — safer for legitimate falsy values like `0` or `''`.